

Frontend Documentation - Virtual Fitting Room

English frontend report covering technologies, implementation decisions, problems faced, and solutions.

1. Frontend Overview

The **Virtual Fitting Room** frontend is built as part of an **ASP.NET Core MVC** application. Its purpose is to give users a clear flow for signing in, uploading a model image and a clothing image, using the camera, and generating an AI-powered virtual try-on result.

The frontend was improved from a generic AI-generated-looking interface into a more practical and professional product experience. The current UI has clearer screens, realistic fashion visuals, consistent controls, and direct user journeys.

2. Frontend Technologies Used

Technology	Usage
Razor Views / CSHTML	Building MVC pages and connecting views with controllers and models.
HTML5	Structuring screens, forms, file inputs, camera elements, and hidden fields.
CSS3	Styling, responsive layouts, visual identity, cards, buttons, and camera views.
Bootstrap 5.3	Grid system, forms, buttons, utilities, and consistent layout behavior.
Bootstrap Icons	Icons for actions such as camera, upload, link, AI fit, and history.
Vanilla JavaScript	Frontend interactivity without adding a separate framework.
MediaDevices API	Opening the browser camera and capturing photos.
Canvas API	Capturing camera frames, resizing images, and drawing overlays.
MediaPipe Pose	Detecting body landmarks and improving garment placement.
Google Fonts	Improving typography using Manrope and Source Sans 3.

3. Main Frontend Files

File	Purpose
VirtualFittingRoom/Views/Shared/_Layout.cshtml	Main layout, fonts, Bootstrap, Bootstrap Icons, and site.css loading.
VirtualFittingRoom/wwwroot/css/site.css	Main visual styling for pages, authentication, studio, upload screen, and responsive layout.
VirtualFittingRoom/wwwroot/js/site.js	Core interaction logic for camera, upload, preview, image resizing, validation, and pose detection.
VirtualFittingRoom/wwwroot/js/live-camera/live-fit-ai.js	Live preview garment placement profiles and fitting calculations.
VirtualFittingRoom/Views/Shared/_TryOnStudio.cshtml	Main studio interface with Live, Files, and Settings sections.
VirtualFittingRoom/Views/TryOn/Upload.cshtml	Upload workflow for model and clothing images.
VirtualFittingRoom/Views/Home/Index.cshtml	Welcome page.
VirtualFittingRoom/Views/Account/login.cshtml	Login page.
VirtualFittingRoom/Views/Registration/Index.cshtml	Registration page.

4. User Experience Structure

Welcome Page

The welcome page introduces the project and provides a direct call to action. A realistic fashion-related background image is used instead of generic illustration-style visuals.

Login and Registration

- Fashion-related visual background.
- Clear glass-style form card.
- Icons inside input fields.
- Show/hide password controls.
- Social login buttons for Google, Facebook, and Apple.

Try-On Studio

- **Live:** Open the camera, view the user, choose a garment image, and run AI Fit.
- **Files:** Upload model and clothing images from the device or use image links.
- **Settings:** Select target body, age, height, weight, clothing type, and model view.

Upload Screen

- Model image upload.
- Clothing image upload.
- Camera capture with countdown.
- Instant image previews.
- Target body, garment type, and fitting area selection.

5. Problems Faced and Solutions

Problem	Impact	Solution
The website looked AI-generated	The UI felt generic and not tailored to the project.	Reworked the visual identity with realistic images, stronger spacing, consistent components, and more professional typography.
User flow was unclear	Users could not easily tell where to start or what to upload.	Split the experience into Live, Files, and Settings with direct action buttons.
Browser camera usage was difficult	Capturing a photo was not obvious or stable.	Used <code>navigator.mediaDevices.getUserMedia</code> with live preview and clear error messages.
Uploaded images could be too large	Large files slowed down upload and processing.	Resized and compressed images in the browser using Canvas before sending them.
Live garment placement was inaccurate	Clothing could appear in the wrong position.	Used MediaPipe Pose landmarks and custom fitting calculations based on shoulders, hips, body area, and garment category.
Different garment types needed different fitting logic	A shirt, pants, dress, abaya, or jumpsuit cannot use the same placement rules.	Added category-to-area mapping for upper, lower, and overall garments.
Some garment choices did not match the selected target body	The experience could feel illogical.	Disabled or hid incompatible options based on Male, Female, and Child targets.
Preview state was unclear	Users could not easily confirm whether images were selected.	Added instant model and clothing previews.
Mobile layout issues	The experience needed better behavior on smaller screens.	Added responsive CSS rules and a mobile-friendly studio layout.
Camera controls could overlap important content	Live controls could cover the camera view.	Organized controls in a fixed tray and added a special Public Live layout.

6. Fixing the AI-Generated Look

One major issue was that the first version of the website looked like a generic AI-generated template rather than a real product. This was solved through several frontend changes:

- Replaced generic visuals with realistic fashion-related imagery.
- Reduced long promotional text and made copy more direct.
- Used a more balanced color system instead of relying on excessive gradients.
- Standardized buttons, inputs, cards, and spacing across pages.
- Added practical UI details such as icons, image previews, camera controls, and bottom navigation.
- Made the studio open directly into a useful camera-based experience instead of only showing a static landing layout.
- Improved responsive behavior so the UI looks intentional on mobile screens.

As a result, the interface became closer to a real usable application rather than a decorative generated page.

7. Key Design Decisions

- A dark visual theme was used to highlight user photos and garment images.
- Glass-style cards were used on authentication screens to create a modern but focused look.
- Icons were added to buttons to make actions easier to understand quickly.
- The studio was designed with a mobile-app-like structure because camera usage is natural on phones.
- Main actions were kept clear: Camera, Garment, Link, and AI Fit.
- The Upload workflow was separated from the Live workflow to support two clear usage paths.

8. JavaScript Functionality

- Show and hide password fields.
- Open and close the camera.
- Capture camera frames and convert them to Base64 images.
- Show instant previews for selected images.
- Resize images before submitting them.
- Load MediaPipe Pose only when needed.
- Store pose landmarks in hidden fields for backend processing.
- Automatically match garment categories to fitting areas.
- Prevent form submission when required images are missing.
- Run AI Fit from the Live screen.
- Open the history modal for previous try-on results.

9. Responsive Design

- CSS Grid and Flexbox layouts.
- Single-column layouts on mobile.
- Adaptive camera and preview heights.
- Fixed bottom navigation inside the studio.
- Touch-friendly Live controls.
- Rules that prevent text and UI elements from overlapping.

10. Backend and AI Integration

The frontend sends data to ASP.NET MVC controllers through forms. Images can be sent as file uploads or as Base64 data captured from the camera. The backend then connects to the virtual try-on service, which can run locally or through Hugging Face / Colab depending on the configuration.

The submitted frontend data includes:

- Model image.
- Clothing image.
- Target body.
- Clothing type.
- Fitting area.
- Model view.
- Pose landmarks when available.

11. Suggested Future Improvements

- Replace browser `alert` messages with toast notifications.
- Add loading states or progress steps while waiting for AI output.
- Improve drag-and-drop upload behavior.
- Add a simple crop tool before image submission.
- Add Arabic / English language switching.
- Add a short onboarding page for first-time users.
- Improve accessibility with more labels and ARIA attributes.

12. Summary

The project frontend evolved from a generic-looking interface into a practical UI that matches the Virtual Fitting Room concept. Razor, Bootstrap, CSS, and JavaScript were used with browser APIs such as Camera, Canvas, and MediaPipe Pose to create a real user experience. The most important improvements were clearer user flow, better visual design, camera support, image previews, live fitting behavior, and responsive mobile usability.